

Audit sicurezza con NetWizard1

Il progetto ha lo scopo di permettere l'attività di valutazione del livello di sicurezza delle informazioni di un'organizzazione da remoto. Il progetto presente su GitHub si può dividere in tre principali rami, i file da installare sui server delle singole organizzazioni, con rispettivo database, i file da installare su un server centrale, che farà da tramite tra noi e le varie organizzazioni e dispone pure di un database centrale contenente tutti i risultati di tutte le scansioni, il terzo ramo sono i file che andremo a utilizzare noi.

Il progetto su Github si trova al seguente indirizzo: <https://github.com/mpozzana3/NetWizard1>

1. Installazioni

Lato cliente, i file utili sono tutti quelli contenuti all'interno della cartella **server**, al suo interno troviamo la cartella **scan**, alcuni file utili alle scansioni, i risultati delle varie scansioni e il file server.py. All'interno del file **scan** troviamo i vari codici scritti in Python per effettuare le scansioni tra cui anche un collegamento ad un altro progetto GitHub: <https://github.com/cddmp/enum4linux-ng> utilizzato per la scansione enum4linux.

Sul lato server, la cartella che interessa a noi è **server-centrale**, al cui interno troviamo file per la configurazione e per la comunicazione con clienti e noi.

La cartella che interessa a noi è **scanner_gui** al cui interno troveremo gli script cli.py per lanciare scansioni e fare analisi database da riga di comando e analisi.py e scansioni.py per lanciare applicazione web con Flask con cui eseguire le medesime operazioni.

All'interno delle cartelle **scan e server-centrale**, troviamo anche i file di configurazione .json per permettere di specificare porta e indirizzo su cui lavoreremo e le info per accedere al DB (queste sono da oscurare con chiavi).

2. Preparazione progetto

Una volta scaricate le rispettive cartelle, clienti e server-centrale troveranno i file creaDB.py per poter creare la struttura del database, le cui info saranno prese dai file di configurazione e possono essere modificate se necessario. Basterà scaricare RDBMS, creare un database e inserire i valori nei specifici

campi di creaDB.py, e lanciare lo script che creerà tutte le tabelle necessarie. I codici sono pensati per database in MariaDB.

```
try:
    # Connessione al primo database
    logging.info("Connessione al primo database...")
    conn1 = mysql.connector.connect(
        host=db_config['host'],
        user=db_config['user'],
        password=db_config['password'],
        database=db_config['name'],
        charset="utf8mb4",
        collation="utf8mb4_general_ci"
    )
    cursor1 = conn1.cursor()
    logging.info("Connessione al primo database completata.")

    # Connessione al secondo database
    logging.info("Connessione al secondo database...")
    conn2 = mysql.connector.connect(
        host=db_config['host'],
        user=db_config['user'],
        password=db_config['password'],
        database="server_centrale",
        charset="utf8mb4",
        collation="utf8mb4_general_ci"
    )
    cursor2 = conn2.cursor()
    logging.info("Connessione al secondo database completata.")
```

Sul server-clienti sarà necessario modificare i dati per collegarsi al DB del server-centrale all'interno del file aggiornadb.py. Modificare il codice, sotto il commento connessione al secondo database.

Sul server-centrale è necessario popolare la cartella aziende all'inizio e successivamente per ogni nuovo cliente su cui si installa il server sonda. Basterà accedere al proprio MariaDB e inserire la query "INSERT INTO azienda (azienda, p_iva, IP_sonda, Porta_sonda) VALUES ('Nome Azienda', 'Partita IVA', 'Indirizzo IP', 'Numero Porta');"

Per eventuali modifiche alla struttura dei database o inserimento/aggiunta di tabelle, oltre ai due file creaDB.py andranno modificati anche i file server/aggiornadb.py e scanner_gui/db_schema.json.

3. Lancio scansione

Una volta creati i due database è tutto pronto per lanciare le scansioni. E' necessario attivare i server lato cliente e su server-centrale. Sul server cliente sarà necessario apporre sudo prima e inserire la pwd poi.

```
[matteo@localhost:~/NetWizard1/server-centrale> python3 server-centrale.py
Server in ascolto su 127.0.0.1:12346...
```

```
[matteo@localhost:~/NetWizard1/server> sudo python3 server.py
[sudo] password di matteo:
Server in ascolto su 0.0.0.0:5003...
Subnet estratta: 172.16.1.0/24
```

```
from flask import Flask, render_template, request, Response, jsonify
import socket
import time

app = Flask(__name__)

SERVER_HOST = "127.0.0.1"
SERVER_PORT = 12346
```

Solo la prima volta che si esegue questo processo sarà necessario modificare i file all'interno di scanner_gui con l'indirizzo corretto per collegarsi al server-centrale. Andiamo all'interno della cartella scanner_gui e apriamo i file (nano <nome-file>) e modifichiamo i campi SERVER_HOST e SERVER_PORT. A questo punto siamo pronti a lanciare l'applicazione web per la scansione.

```
[matteo@localhost:~/NetWizard1] cd scanner_gui
[matteo@localhost:~/NetWizard1/scanner_gui] ls
analisi.py db_schema.json __pycache__ scansioni.py templates
[matteo@localhost:~/NetWizard1/scanner_gui] python3 scansioni.py
* Serving Flask app 'scansioni'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5005
* Running on http://172.16.1.141:5005
Press CTRL+C to quit
* Restarting with watchdog (inotify)
* Debugger is active!
* Debugger PIN: 141-564-906
```

Come possiamo notare l'applicazione sarà disponibile all'indirizzo <http://172.16.1.141:5005>, che ovviamente può essere cambiato modificando il file scansioni.py. Se tutto funziona come dovrebbe accedendo all'indirizzo sopra si presenterà una pagina per inserire i dati per la nostra scansione. Come possiamo notare dalla foto sotto ci verranno fornite tutte le opzioni di scansione disponibile, una volta compilati i campi Azienda, p_iva e tipo scansione possiamo cliccare invia per inviare i dati al server-centrale.

The screenshot shows a web application interface with a light gray background. At the top, there is a blue header with the text "Che tipo di scansione vuoi fare?". Below this, there is a black rectangular box containing a list of scan types in green text: ARP_PASSIVA, ARP_ATTIVA, NMAP, NBTSCAN, ENUM4LINUX (solo dopo NBTSCAN), SMBMAP (solo dopo NBTSCAN), SMBCLIENT (solo dopo NBTSCAN), HASSCAN, and COMPLETA. Below the black box, there is a blue header with the text "Scansioni di rete". Underneath, there are three white input fields with blue borders. The first field contains the text "GFTECH", the second field contains "IT123123123", and the third field contains "COMPLETA". Below the input fields, there is a blue button with the text "Invia". At the bottom of the interface, there is a black horizontal bar.

Il server centrale li riceve, interroga il database per vedere se è presente l'azienda indicata con la rispettiva p_iva, se sì estrapola indirizzo e porta per collegarsi al server del cliente, altrimenti ci comunica l'errore di inserimento. Una volta collegatosi al server del cliente il server-centrale girerà il valore del tipo di scansione richiesto.

```
[matteo@localhost:~/NetWizard1/server> sudo python3 server.py
[sudo] password di matteo:
Server in ascolto su 0.0.0.0:5003...
Subnet estratta: 172.16.1.0/24
Connessione ricevuta da ('127.0.0.1', 39450)
Scelta scansione ricevuta: COMPLETA
Scelta scansione registrata nel database con ID: 30
Lancio una scansione COMPLETA con l'ID: 30
```

Se il tipo di scansione inserito è errato (non presente o non valido), il server lo comunica al server-centrale che girerà il messaggio a noi, altrimenti lancia la scansione scelta estrapolando un id_scansione progressivo dalla sua tabella scansioni.

Si potrebbe notare accedendo al DB del server cliente che una scansione COMPLETA è stata lanciata con id_scansione 30 e non è ancora terminata (stato 1)

```
[MariaDB [test]> SELECT * FROM scansioni WHERE id_scansione=30;;
+-----+-----+-----+-----+-----+
| id_scansione | p_iva      | timestamp           | tipo_scansione | stato |
+-----+-----+-----+-----+-----+
| 30           | IT12345679810 | 2025-02-17 14:47:36 | COMPLETA        | 1     |
+-----+-----+-----+-----+-----+
1 row in set (0.007 sec)
```

Una volta terminata la scansione, il server cliente aggiornerà lo stato a 0, e lancerà lo script aggiornadb.py che aggiorna il database del server centrale con gli ultimi valori ottenuti, e comunica la riuscita o meno di questi passaggi al server-centrale che li girerà a noi.

```
[matteo@localhost:~/NetWizard1/server> sudo python3 server.py
[sudo] password di matteo:
Server in ascolto su 0.0.0.0:5003...
Subnet estratta: 172.16.1.0/24
Connessione ricevuta da ('127.0.0.1', 39450)
Scelta scansione ricevuta: COMPLETA
Scelta scansione registrata nel database con ID: 30
Lancio una scansione COMPLETA con l'ID: 30
Output della scansione con ID 30 aggiornato o inserito nella tabella stdout.
Stato della scansione con ID 30 aggiornato a COMPLETATA.
Scansione COMPLETA terminata per l'ID 30
Output della scansione con ID 30 aggiornato o inserito nella tabella stdout.
Script aggiornadb.py eseguito.
Ho chiuso la connessione.
```

Che tipo di scansione vuoi fare?

```
ARP_PASSIVA
ARP_ATTIVA
NMAP
NBTSCAN
ENUMALINUX (solo dopo NBTSCAN)
SMBMAP (solo dopo NBTSCAN)
SMBCLIENT (solo dopo NBTSCAN)
MASSCAN
COMPLETA
```

Scansioni di rete

GFTECH

IT123123123

COMPLETA

Invia

📧 Risposta ricevuta: Collegamento riuscito. Ti sei collegato alla sonda dell'azienda GFTECH.

📧 Risposta ricevuta: Lancio una scansione COMPLETA con l'ID: 30

📧 Risposta ricevuta: Scansione COMPLETA terminata per l'ID 30.

📧 Risposta ricevuta: Script aggiornadb.py eseguito con successo. I dati sono stati aggiornati nel database centrale.

4. Analisi risultati

Una volta termina la scansione si può terminare il server del cliente, e anche l'applicazione web. A questo punto se si vuole analizzare i risultati delle scansioni basterà, all'interno della stessa cartella, lanciare il file analisi.py.

```
matteo@localhost:~/NetWizard1> cd scanner_gui
matteo@localhost:~/NetWizard1/scanner_gui> ls
analisi.py db_schema.json __pycache__ scansioni.py templates
matteo@localhost:~/NetWizard1/scanner_gui> python3 analisi.py
* Serving Flask app 'analisi'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5006
* Running on http://172.16.1.141:5006
Press CTRL+C to quit
* Restarting with watchdog (inotify)
* Debugger is active!
* Debugger PIN: 141-564-906
```

Se non si è terminato il server-centrale, in automatico si collegherà, altrimenti basterà lanciarlo di nuovo come fatto sopra. A questo punto possiamo notare che è attivo l'indirizzo <http://172.16.1.141.5006>. Basterà accedervi per visualizzare una pagina che ci permette di specificare una tabella, una o più rispettive colonne (o tutte selezionando Seleziona tutte), ed eventuali vincoli.

Analisi Dati

Seleziona la Tabella:
nmap

Seleziona le Colonne:

☐ Seleziona tutte

☒ id_scansione

☒ ip

☐ timestamp

☐ vendor

☒ hostname

☒ extraports_count

☒ extraports_state

☒ port_id

☐ port_state

☐ port_service_name

☐ port_product

☐ port_script_id

☐ port_script_output

☐ p_iva

Inserisci un vincolo (opzionale):
id_scansione=301

Invia

Fatto ciò invieremo la query al server-centrale che interrogherà il proprio DB grazie ad uno script interno e ci girerà i risultati.

Inserisci un vincolo (opzionale):

Query Eseguita:

SELECT id_scansione, ip, hostname, extraports_count, extraports_state, port_id FROM nmap WHERE id_scansione='30'

Risultati

Show
5
entries

Search:

id_scansione	ip	hostname	extraports_count	extraports_state	port_id
30	172.16.1.1	None	997	closed	22
30	172.16.1.1	None	997	closed	2000
30	172.16.1.1	None	997	closed	8291
30	172.16.1.2	fs02.gfttech.srl	996	closed	22
30	172.16.1.2	fs02.gfttech.srl	996	closed	139

Showing 1 to 5 of 2,448 entries

Previous12345...490Next

Esporta come TXT

Esporta come Immagine

Esporta come CSV

Esporta come PDF

Nuova Analisi

Come possiamo vedere, l'applicazione web ci mostrerà i risultati in forma tabellare, dandoci l'opportunità di selezionare il numero di righe a schermo (in alto a sinistra) e anche di cercare valori chiave (in alto a destra). Sotto ci mostrerà il numero di righe e le pagine.

Abbiamo varie possibilità, esportare come testo (utile per i file scansione completi all'interno della tabella scansioni per query come `SELECT enum4json FROM file_scansioni WHERE...` per estrapolare il file json di una determinata scansione), esportare come immagine (cattura lo screenshot della tabella a schermo), come csv (utile per tabelle grandi ma con celle poco popolate). Cliccando su "Nuova Analisi" ritorneremo alla pagina di prima dove potremo inserire nuovi dati per una nuova analisi.

4b Nmap

Uno dei file più utili è sicuramente l'analisi fatta lanciando nmap. Questa oltre che estrapolata nella tabella nmap si può trovare completa all'interno della tabella file_scansioni (anche in formato xml). Basta interrogare il db tramite file analisi.py sulla tabella file_scansioni sulla colonna nmaphtml e scaricare il risultato in formato txt. A questo punto basta eliminare la prima riga che conterrà il titolo nmaphtml, e salvare il file con formato *.html. A questo punto aprendo il file si aprirà una comodissima pagina web per analizzare la scansione nmap.

Query Eseguita:

SELECT nmaphtml FROM file_scansioni WHERE id_scansione='30'

Risultati

Show
5
entries

Search:

nmaphtml
<code>DataTable(;;); \$("#table-overview").DataTable({ "lengthMenu": [[5, 10, 25, 50, 100, -1], [5, 10, 25, 50, 100, "All"]], "columnDefs": [{ "targets": [1], "type": "ip-address" },], }); </script> </body> </html></code>

Showing 1 to 1 of 1 entries

Previous1Next

Esporta come TXT

Esporta come Immagine

Esporta come CSV

Esporta come PDF

Nuova Analisi

5. Utilizzo riga di comando

Se si preferisce lavorare da riga di comando senza dover aprire applicazioni web basterà lanciare cli.py all'interno di **scanner_gui**. Lo script si collegherà al server-centrale e server-cliente se attivi e tramite domande interattive ci guiderà verso una scansione o analisi del database.

```
~ ssh matteo@172.16.1.141
matteo@localhost:~/NetWizard/scanner_gui> ls
analisi.py  cli.py  db_schema.json  __pycache__  scansioni.py  templates
matteo@localhost:~/NetWizard/scanner_gui> python3 cli.py
Benvenuto, provo a collegarmi al server-centrale...
Sei riuscito a collegarti al server-centrale. Cosa vuoi fare?
Scegli tra: 1. Analisi 2. Scansioni
scansioni
Hai scelto scansioni. Inserisci le informazioni per la scansione.
Azienda: GFTECH
Partita IVA: IT123123123
Scansioni disponibili: ARP_PASSIVA, ARP_ATTIVA, NMAP, NBTSCAN, ENUM4LINUX, SMBMAP, SMBCLIENT, MASSCAN, COMPLETA
Scansione: NBTSCAN
Ricevuto dal server [1]: Collegamento riuscito. Ti sei collegato alla sonda dell'azienda GFTECH.
Ricevuto dal server [2]: Lancio una scansione NBTSCAN con l'ID: 34
Ricevuto dal server [3]: Scansione NBTSCAN terminata per l'ID 34.
Ricevuto dal server [4]: Script aggiornato.py eseguito con successo. I dati sono stati aggiornati nel database centrale.
matteo@localhost:~/NetWizard/scanner_gui>

~ ssh matteo@172.16.1.141
matteo@localhost:~/NetWizard/scanner_gui> python3 cli.py
Benvenuto, provo a collegarmi al server-centrale...
Sei riuscito a collegarti al server-centrale. Cosa vuoi fare?
Scegli tra: 1. Analisi 2. Scansioni
analisi
Tabelle disponibili: ['aziende', 'nbtscan', 'enum4linux', 'extended_enum', 'masscan', 'tabella_host', 'smbclient', 'smbmap', 'snmp_scan', 'scansioni', 'stdout', 'file_scansioni', 'nmap']
Tabella da analizzare: aziende
Colonne nella tabella: ['id', 'azienda', 'p_iva', 'ip_sonda', 'porta_sonda']
Scegli colonne da analizzare, usa virgola per separare (* per tutte): *
+ Aggiungi vincolo opzionale (premi INVIO per lasciare vuoto)
Vincolo opzionale (es. id_scansione='1'): azienda='GFTECH'
Invio query al server: SELECT * FROM aziende WHERE azienda='GFTECH'
Ricevuto 168 byte. Totale ricevuto: 168 byte.
Risposta completa ricevuta (168 byte).
Risultati della query:
id $ azienda $ p_iva $ ip_sonda $ porta_sonda $
----- $
1 $ GFTECH $ IT123123123 $ 0.0.0.0 $ 5003 $
matteo@localhost:~/NetWizard/scanner_gui>
```

6. Errori

Di base gli script gestiscono e informano su tutti i tipi di errori quando si cerca di iniziare una scansione, da collegamenti non riusciti, inserimenti errati del tipo scansione o nome azienda e altro. Ma se una scansione non dovesse andare a buon fine una volta lanciata (a fine scansione lo stato dovrebbe rimanere 1) o ci sembra che qualcosa sia andato storto, tramite l'id_scansione possiamo fare un controllo. Infatti sul database è presente anche la tabella stdout, che contiene tutti gli stdout del server-cliente per ogni scansione.

Query Eseguita:

```
SELECT id_scansione, stdout, p_iva FROM stdout WHERE id_scansione='30'
```

Risultati

Show

5 entries

Search:

id_scansione	stdout	p_iva
30	Subnet estratta: 172.16.1.0/24 Lancio scan/scan-1.py... scan/sca	IT12345679810

Showing 1 to 1 of 1 entries

Previous1Next

Esporta come TXT

7. Scansioni

Di base i codici per lanciare le scansioni sono già pronti così ed eseguono le scansioni più complete possibili, però se si vuole un particolare tipo di scansione, o magari più specifiche e meno pesanti bisognerà modificare i file all'interno della cartella **scan** nei server dei clienti.

Per fare un esempio, masscan lista porte TCP/UDP disponibili nello spazio di indirizzamento pubblico, accedendo al file scan/masscan.py si può modificare il range di porte da scandire, più ampio ovviamente significa anche più tempo.

```
GNU nano 8.3 masscan.py
    return json.load(f)

def run_masscan(target_ip):
    commands = [
        ["masscan", "-oX", "masscan.xml", "--ports", "0-500", target_ip],
        ["masscan", "-oX", "masscanu.xml", "--ports", "U:0-500", target_ip]
    ]
    for cmd in commands:
```

8. Database

I database sul server e sul server-centrale sono pensati per essere pressoché identici per semplicità di trasferimento dei dati.

Sul server ogni scansione lanciata verrà inserita nella tabella scansioni con un nuovo id_scansione progressivo (30 nel nostro esempio), e questo valore potrà essere usato come identificatore per analizzare le altre tabelle.

Due saranno le differenze del DB su server-centrale, una tabella aziende con le info per collegarsi al server sonda, e una colonna in più "p_iva" per ogni tabella. Ad ogni aggiornamento dei dati, un server sonda passerà i valori delle nuove scansioni, affiancati dalla p_iva dell'azienda che rappresenta. Così che su ogni tabella del db potremmo trovare id_scansione duplicati ma la coppia (p_iva, id_scansione) sarà unica.

Sul file schema_db.json dentro scanner_gui c'è la struttura del DB usata dagli script delle web app per semplificare la creazione di query. Quindi nel caso di modifica al db sarà necessario modificare anche quello.

9. Miglioramenti

Alcune cose che sono da migliorare o non sono riuscito a implementare.

Sicuramente si può migliorare le web app per ottenere un'interazione soprattutto con i risultati tabellari ancora migliore.

Per quanto riguarda la scansione SNMP è presente nel progetto ma non fornendo risultati nei test di scansione fatti in azienda non ho potuto ne effettivamente verificare l'output della scansione ne il corretto funzionamento del codice python. Ho provato a basare la struttura della tabella e raccolta informazione da esempi di output sul web.

In questo momento i log che riceviamo dopo aver lanciato una scansione, sono i 4 nella foto sopra, mentre per tutto il resto serve accedere alla tabella stdout. Però si potrebbe volere ricevere molte più informazioni sull'andamento della scansione, allora sarebbe da modificare i tre file (server.py, server-centrale.py, cli.py) per far si che comunichino più messaggi.

Per ora gli script non mi hanno dato problemi nel scambio di messaggi nonostante la lunghezza dei risultati di alcune query.